

CL-2006 Operating System Lab

Code Explanation Guide

Lab 09: CPU Scheduling Algorithms FCFS · SJF · SRTF · Round Robin

*Complete Line-by-Line Code Walkthrough
with Manual Calculations & Quiz Preparation*

Given Codes (3):

FCFS Base Code · SJF Base Code · Round Robin Base Code

Task Codes (6):

Task 1: FCFS with User Input & Averages

Task 2: SJF with User Input & Averages

Task 3: SRTF (Preemptive SJF) with User Input & Averages

Task 4: Round Robin TQ=4 with User Input & Averages

Task 5: RR Version 1 – No Ready Queue (Analysis)

Task 6: RR Version 2 – With Proper Ready Queue

Contents

1	Key Theoretical Concepts – CPU Scheduling	2
1.1	Why CPU Scheduling?	2
1.2	The Three Key Metrics – Know These for Every Algorithm	2
1.3	Algorithm Comparison Table	2
2	Given Code 1: FCFS Base Code	3
2.1	Line-by-Line Explanation	4
3	Given Code 2: SJF Base Code	6
3.1	Line-by-Line Explanation	7
4	Given Code 3: Round Robin Base Code	10
4.1	Line-by-Line Explanation	11
5	Task 1: FCFS – User Input + Averages (task1_fcfs.c)	14
5.1	Line-by-Line Explanation	15
6	Task 2: SJF – User Input + Averages (task2_sjf.c)	17
6.1	Line-by-Line Explanation (New/Key Lines Only)	19
7	Task 3: SRTF – Preemptive SJF (task3_srtf.c)	20
7.1	Line-by-Line Explanation (Key SRTF-Specific Lines)	22
8	Task 4: Round Robin TQ=4 – User Input (task4_rr_tq4.c)	24
8.1	Line-by-Line Explanation	26
9	Task 5: RR Version 1 – Analysis of No-Queue Problem	28
9.1	What is Wrong with Version 1?	28
10	Task 6: RR Version 2 – With Proper Ready Queue (task6_rr_v2_queue.c)	29
10.1	Line-by-Line Explanation	32
11	Quick Reference – Quiz Cheat Sheet	35
11.1	Formulas	35
11.2	Algorithm Differences in Code	35
11.3	Common Mistakes to Avoid	35
11.4	Files Summary	36

Key Theoretical Concepts – CPU Scheduling

Why CPU Scheduling?

OS Concept – Why Scheduling Exists

In a multiprogramming OS, many processes are ready to run at the same time but only ONE process can use the CPU at a time. The **CPU scheduler** decides which process gets the CPU next and for how long. The goal is to maximize CPU utilization, minimize waiting time, and ensure fairness.

The Three Key Metrics – Know These for Every Algorithm

Metric		Formula	Meaning
Completion Time (CT)	Time	Time process finishes	When it left the CPU for good
Turnaround Time (TAT)	Time	$TAT = CT - AT$	Total time from arrival to finish
Waiting Time (WT)		$WT = TAT - BT$	Time spent waiting in ready queue

Average Formulas

$$\text{Avg TAT} = \frac{TAT_1 + TAT_2 + \dots + TAT_n}{n} \quad \text{Avg WT} = \frac{WT_1 + WT_2 + \dots + WT_n}{n}$$

Algorithm Comparison Table

Algorithm	Preemptive?	Selection Basis	Key Property
FCFS	No	Arrival order	Simple; may cause convoy effect
SJF (SPN)	No	Shortest BT among arrived	Optimal avg WT (non-preemptive)
SRTF	Yes	Shortest re-maining BT	Can preempt current process
Round Robin	Yes	Circular with time quantum	Fair; each gets equal CPU slices

Given Code 1: FCFS Base Code

What This Code Does

Implements First Come First Served scheduling for 3 fixed processes. Processes run in arrival order. Calculates CT, TAT, WT for each process.

Manual Calculation in the Code Comment

P	AT	BT
1	0	4
2	1	3
3	2	2

t=0 -> P1 runs 4 -> CT=4

t=4 -> P2 runs 3 -> CT=7

t=7 -> P3 runs 2 -> CT=9

TAT: 4-0=4 ; 7-1=6 ; 9-2=7

WT: 4-4=0 ; 6-3=3 ; 7-2=5

```

1  #include <stdio.h>
2
3  int main() {
4
5      int n = 3;
6
7      int pid[] = {1, 2, 3};
8      int at[] = {0, 1, 2};
9      int bt[] = {4, 3, 2};
10
11     int ct[3], tat[3], wt[3];
12
13     int i;
14     int current_time = 0;
15
16     for (i = 0; i < n; i++) {
17
18         if (current_time < at[i]) {
19             current_time = at[i];
20         }
21
22         current_time += bt[i];
23         ct[i] = current_time;
24     }
25
26     for (i = 0; i < n; i++) {
27         tat[i] = ct[i] - at[i];
28         wt[i] = tat[i] - bt[i];
29     }
30

```

```
31     printf("P\tAT\tBT\tCT\tTAT\tWT\n");
32
33     for (i = 0; i < n; i++) {
34         printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
35             pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
36     }
37
38     return 0;
39 }
```

Listing 1: FCFS Base Code – Given in Lab

Line-by-Line Explanation

```
int n = 3;
```

Number of processes is hardcoded to 3. In Task 1, this becomes user input.

```
int pid[] = {1, 2, 3};
```

Array of process IDs. Index 0 = P1, index 1 = P2, index 2 = P3. Used only for display purposes.

```
int at[] = {0, 1, 2};
```

Arrival Times array. P1 arrives at time 0, P2 at time 1, P3 at time 2. FCFS assumes these are sorted by arrival time.

```
int bt[] = {4, 3, 2};
```

Burst Times array. P1 needs 4 units, P2 needs 3, P3 needs 2. These are the CPU execution times.

```
int ct[3], tat[3], wt[3];
```

Three output arrays: **ct** = Completion Time, **tat** = Turnaround Time, **wt** = Waiting Time. One value per process.

```
int current_time = 0;
```

A simulated clock. Starts at 0 and advances as processes execute. This variable tracks “what time is it right now on the CPU timeline”.

```
for (i = 0; i < n; i++) {
```

Loop through all processes in order (index 0 to n-1). Since FCFS runs processes in arrival order and the array is sorted by arrival time, this loop gives the correct execution order.

```
if (current_time < at[i]) { current_time = at[i]; }
```

Handles the **CPU idle gap**. If the current process hasn’t arrived yet (its arrival time is in the future), the CPU must wait. We jump the clock forward to the process’s

arrival time. Example: if `current_time=2` but next process arrives at `t=5`, we set `current_time=5`.

```
current_time += bt[i];
```

The process runs for its full burst time. We add burst time to the clock. After this line, `current_time` represents when this process finishes.

```
ct[i] = current_time;
```

Record the completion time of process `i`. This is the exact moment the process finishes on the CPU.

```
tat[i] = ct[i] - at[i];
```

Turnaround Time = Completion Time - Arrival Time. This is the total time the process spent in the system (waiting + executing).

```
wt[i] = tat[i] - bt[i];
```

Waiting Time = TAT - Burst Time. Since TAT includes both waiting and execution, subtracting execution time gives us only the waiting time.

```
printf("P\tAT\tBT\tCT\tTAT\tWT\n");
```

Prints the header row of the results table. `\t` is a tab character for column alignment.

Expected Output

P	AT	BT	CT	TAT	WT
P1	0	4	4	4	0
P2	1	3	7	6	3
P3	2	2	9	7	5

Given Code 2: SJF Base Code

What This Code Does

Implements Shortest Job First (non-preemptive) for 4 fixed processes. At each scheduling decision point, selects the **already-arrived** process with the **shortest burst time**. Uses a `done[]` array to track completed processes.

Manual Calculation in the Code Comment

P	AT	BT
1	0	6
2	2	4
3	3	2
4	5	1

t=0: Only P1 arrived -> run P1(6) -> CT[0]=6
 t=6: P2(4), P3(2), P4(1) all arrived -> pick P4(BT=1) -> CT[3]=7
 t=7: P2(4), P3(2) -> pick P3(BT=2) -> CT[2]=9
 t=9: P2(4) only -> run P2(4) -> CT[1]=13

TAT: 6-0=6 ; 13-2=11 ; 9-3=6 ; 7-5=2

WT: 6-6=0 ; 11-4=7 ; 6-2=4 ; 2-1=1

```

1 #include <stdio.h>
2
3 int main() {
4
5     int n = 4;
6
7     int pid[] = {1, 2, 3, 4};
8     int at[] = {0, 2, 3, 5};
9     int bt[] = {6, 4, 2, 1};
10
11     int ct[n], tat[n], wt[n];
12     int done[n];
13
14     int i;
15
16     int completed = 0, current_time = 0;
17     int smallest, flag;
18
19     for (i = 0; i < n; i++) {
20         done[i] = 0;
21     }
22
23     while (completed < n) {
24
25         smallest = -1;
26         flag = 0;
27

```

```

28     for (i = 0; i < n; i++) {
29
30         if (!done[i] && at[i] <= current_time) {
31
32             if (smallest == -1 || bt[i] < bt[smallest]) {
33                 smallest = i;
34                 flag = 1;
35             }
36         }
37     }
38
39     if (!flag) {
40         current_time++;
41         continue;
42     }
43
44     current_time += bt[smallest];
45     ct[smallest] = current_time;
46     done[smallest] = 1;
47     completed++;
48 }
49
50 for (i = 0; i < n; i++) {
51     tat[i] = ct[i] - at[i];
52     wt[i] = tat[i] - bt[i];
53 }
54
55 printf("P\tAT\tBT\tCT\tTAT\tWT\n");
56
57 for (i = 0; i < n; i++) {
58     printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
59         pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
60 }
61
62 return 0;
63 }

```

Listing 2: SJF Base Code – Given in Lab

Line-by-Line Explanation

```
int done[n];
```

A boolean array where `done[i] = 0` means process `i` is not yet completed, and `done[i] = 1` means it has finished. This is needed because SJF does not execute processes in array order – it picks the shortest, so we need to skip already-finished ones.

```
int completed = 0, current_time = 0;
```

Two counters: **completed** tracks how many processes have finished (loop until this

equals `n`), **current_time** is our simulated clock.

```
int smallest, flag;
```

Two helper variables: **smallest** holds the array index of the process with the shortest burst time so far in the search, **flag** is set to 1 if we found at least one eligible process this round.

```
for (i = 0; i < n; i++) { done[i] = 0; }
```

Initialize all entries in `done[]` to 0 (no process is done yet). Must be done before the scheduling loop.

```
while (completed < n) {
```

Main scheduling loop. Continues until every process has been completed. Each iteration of this loop executes exactly ONE process to completion.

```
smallest = -1; flag = 0;
```

Reset search variables at the start of each scheduling decision. **smallest** = -1 is a sentinel meaning “haven’t found any eligible process yet”.

```
for (i = 0; i < n; i++) {
```

Inner loop: scan all `n` processes to find the best candidate for execution.

```
if (!done[i] && at[i] <= current_time) {
```

Two conditions must BOTH be true:

!done[i] – the process hasn’t finished yet

at[i] <= current_time – the process has already arrived

If both true, this process is eligible for CPU.

```
if (smallest == -1 || bt[i] < bt[smallest]) { smallest = i; flag = 1; }
```

If this is the first eligible process found (**smallest** == -1), select it. OR if this process has a shorter burst time than the current best, update **smallest** to this index. This finds the minimum BT among all arrived, unfinished processes.

```
if (!flag) { current_time++; continue; }
```

If **flag** is still 0, no eligible process was found (all remaining processes haven’t arrived yet). Advance the clock by 1 (CPU idle) and restart the while loop.

```
current_time += bt[smallest];
```

Execute the selected process to completion (non-preemptive). Add its full burst time to the clock.

```
ct[smallest] = current_time;
```

Record its completion time.

```
done[smallest] = 1; completed++;
```

Mark it as done and increment the completed counter.

Expected Output

P	AT	BT	CT	TAT	WT
P1	0	6	6	6	0
P2	2	4	13	11	7
P3	3	2	9	6	4
P4	5	1	7	2	1

OS Concept – Why SJF is NOT Used in Real Systems

SJF requires knowing the burst time of every process IN ADVANCE – which is impossible in practice. In real operating systems, the scheduler uses an **exponential average** of past CPU bursts to predict the next burst time. SRTF (preemptive SJF) is closer to what real systems approximate.

Given Code 3: Round Robin Base Code

What This Code Does

Implements Round Robin scheduling for 3 fixed processes with a fixed time quantum (TQ=3). Each process gets at most TQ units of CPU before being preempted and put back in the queue.

Manual Calculation in the Code Comment

P	AT	BT	TQ=3
1	0	5	
2	2	1	
3	4	3	

t=0: P1 arrives (rem=5) -> run P1 for 3 -> rem[0]=2, t=3

t=3: P2(rem=1), P3 not yet; run P2 for 1 -> done, CT[1]=4, t=4

t=4: P3(rem=3) -> run P3 for 3 -> done, CT[2]=7, t=7

t=7: P1(rem=2) -> run P1 for 2 -> done, CT[0]=9, t=9

CT: P1=9, P2=4, P3=7

TAT: 9-0=9 ; 4-2=2 ; 7-4=3

WT: 9-5=4 ; 2-1=1 ; 3-3=0

```

1 #include <stdio.h>
2
3 int main() {
4
5     int n = 3;
6     int pid[] = {1, 2, 3};
7     int at[] = {0, 2, 4};
8     int bt[] = {5, 1, 3};
9
10    int tq = 3;
11
12    int remaining[3];
13    int ct[3], tat[3], wt[3];
14
15    int completed = 0;
16    int current_time = 0;
17    int i;
18
19    for (i = 0; i < n; i++) {
20        remaining[i] = bt[i];
21    }
22
23    while (completed < n) {
24
25        int executed = 0;
26
27        for (i = 0; i < n; i++) {

```

```

28
29         if (remaining[i] > 0 && at[i] <= current_time) {
30
31             executed = 1;
32
33             if (remaining[i] > tq) {
34                 current_time += tq;
35                 remaining[i] -= tq;
36             }
37             else {
38                 current_time += remaining[i];
39                 ct[i] = current_time;
40                 remaining[i] = 0;
41                 completed++;
42             }
43         }
44     }
45
46     if (!executed) {
47         current_time++;
48     }
49 }
50
51 for (i = 0; i < n; i++) {
52     tat[i] = ct[i] - at[i];
53     wt[i] = tat[i] - bt[i];
54 }
55
56 printf("P\tAT\tBT\tCT\tTAT\tWT\n");
57
58 for (i = 0; i < n; i++) {
59     printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
60         pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
61 }
62
63 return 0;
64 }

```

Listing 3: Round Robin Base Code – Given in Lab (TQ=3)

Line-by-Line Explanation

```
int tq = 3;
```

The Time Quantum. Each process can run for at most 3 time units before being preempted. If a process has less than 3 units remaining, it runs until done.

```
int remaining[3];
```

Stores the **remaining burst time** for each process. Initially set to the full burst time. This decreases each time the process uses the CPU. When it reaches 0, the

process is done.

```
for (i = 0; i < n; i++) { remaining[i] = bt[i]; }
```

Initialize remaining burst times equal to the original burst times. This is done BEFORE the scheduling loop.

```
while (completed < n) {
```

Main loop: runs until all n processes complete. Unlike FCFS and SJF, one iteration here does NOT necessarily complete a process – it might just use one quantum.

```
int executed = 0;
```

Flag to detect CPU idle. Set to 1 if any process ran in this pass through the for loop. If it stays 0 after the full for loop, no process was eligible and we advance the clock.

```
if (remaining[i] > 0 && at[i] <= current_time) {
```

Two conditions:

remaining[i] > 0 – process still has work to do

at[i] <= current_time – process has arrived

Only then can this process get the CPU.

```
executed = 1;
```

Mark that at least one process ran this round. This prevents the idle clock-advance at the end.

```
if (remaining[i] > tq) {
```

If the process needs MORE than one quantum to finish, it will be preempted.

```
current_time += tq; remaining[i] -= tq;
```

The process runs for exactly TQ time units. Clock advances by TQ. Remaining burst decreases by TQ. The process is NOT done – it will get another turn later.

```
else {
```

If remaining burst time is LESS THAN OR EQUAL TO the time quantum, the process can finish in this quantum.

```
current_time += remaining[i]; ct[i] = current_time; remaining[i] = 0;
completed++;
```

The process finishes. Clock advances by its remaining time. Completion time is recorded. Remaining set to 0. Completed counter incremented.

```
if (!executed) { current_time++; }
```

If the entire for loop ran without any process executing (all remaining processes haven't arrived yet), advance the clock by 1 to handle the CPU idle period.

Expected Output

P	AT	BT	CT	TAT	WT
P1	0	5	9	9	4
P2	2	1	4	2	1
P3	4	3	7	3	0

This RR Has No Proper Ready Queue

This implementation scans processes from index 0 every round. It does NOT maintain a real FIFO ready queue. This means the scheduling order depends on array indices, not on when processes actually arrive and join the queue. Task 6 fixes this with a proper queue implementation.

Task 1: FCFS – User Input + Averages (task1_fcfs.c)

Task 1 Objective

Extend the FCFS base code to: (1) accept n, AT, and BT from the user at runtime, (2) calculate and display Average TAT and Average WT, and (3) include a manual calculation comment for n=5.

Manual Calculation for n

P	AT	BT
1	0	4
2	1	3
3	2	5
4	4	2
5	6	1

Execution (FCFS = arrival order):

t=0 -> P1(4) -> CT[0]=4

t=4 -> P2(3) -> CT[1]=7

t=7 -> P3(5) -> CT[2]=12

t=12 -> P4(2) -> CT[3]=14

t=14 -> P5(1) -> CT[4]=15

TAT: 4-0=4 ; 7-1=6 ; 12-2=10 ; 14-4=10 ; 15-6=9

WT: 4-4=0 ; 6-3=3 ; 10-5=5 ; 10-2=8 ; 9-1=8

Avg TAT = (4+6+10+10+9)/5 = 39/5 = 7.80

Avg WT = (0+3+5+8+8)/5 = 24/5 = 4.80

```

1 #include <stdio.h>
2
3 int main() {
4
5     int n;
6
7     printf("Enter number of processes: ");
8     scanf("%d", &n);
9
10    int pid[n], at[n], bt[n];
11    int ct[n], tat[n], wt[n];
12    int i;
13
14    for (i = 0; i < n; i++) {
15        pid[i] = i + 1;
16        printf("Enter Arrival Time and Burst Time for P%d: ",
17              pid[i]);
18        scanf("%d %d", &at[i], &bt[i]);
19    }

```

```

20     int current_time = 0;
21
22     for (i = 0; i < n; i++) {
23
24         if (current_time < at[i]) {
25             current_time = at[i];
26         }
27
28         current_time += bt[i];
29         ct[i] = current_time;
30     }
31
32     for (i = 0; i < n; i++) {
33         tat[i] = ct[i] - at[i];
34         wt[i] = tat[i] - bt[i];
35     }
36
37     printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
38     for (i = 0; i < n; i++) {
39         printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
40             pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
41     }
42
43     float sum_tat = 0, sum_wt = 0;
44     for (i = 0; i < n; i++) {
45         sum_tat += tat[i];
46         sum_wt += wt[i];
47     }
48
49     printf("\nAverage TAT = %.2f\n", sum_tat / n);
50     printf("Average WT = %.2f\n", sum_wt / n);
51
52     return 0;
53 }

```

Listing 4: task1_fcfs.c – FCFS with User Input and Averages

Line-by-Line Explanation

```
int n; printf("Enter number of processes: "); scanf("%d", &n);
```

Asks the user to enter how many processes to schedule. `scanf("%d", &n)` reads the integer from keyboard and stores it in `n`. This replaces the hardcoded `n = 3` from the base code.

```
int pid[n], at[n], bt[n]; int ct[n], tat[n], wt[n];
```

Declares arrays with **variable length** (VLA – Variable Length Array in C99). The size is `n`, which was just read from the user. This is valid in C99 and later.


```
for (i = 0; i < n; i++) { pid[i] = i+1; printf(...); scanf("%d %d",
&at[i], &bt[i]); }
```

Input loop: for each process, automatically assigns PID as i+1, then asks the user to type the arrival time and burst time on one line separated by a space. `scanf("%d %d", &at[i], &bt[i])` reads two integers at once.

```
float sum_tat = 0, sum_wt = 0;
```

Declares floating-point accumulators for the sum of TATs and WTs. Using `float` instead of `int` so the division gives a decimal result.

```
for (i = 0; i < n; i++) { sum_tat += tat[i]; sum_wt += wt[i]; }
```

Accumulates the sum of all TAT and WT values. `+=` is shorthand for `sum_tat = sum_tat + tat[i]`.

```
printf("\nAverage TAT = %.2f\n", sum_tat / n);
```

Divides the total TAT by `n` to get the average. `%.2f` formats the float to exactly 2 decimal places. The division is float division because `sum_tat` is a float.

Sample Run with n=5 (matching manual)

```
Enter number of processes: 5
Enter AT and BT for P1: 0 4
Enter AT and BT for P2: 1 3
Enter AT and BT for P3: 2 5
Enter AT and BT for P4: 4 2
Enter AT and BT for P5: 6 1
```

P	AT	BT	CT	TAT	WT
P1	0	4	4	4	0
P2	1	3	7	6	3
P3	2	5	12	10	5
P4	4	2	14	10	8
P5	6	1	15	9	8

```
Average TAT = 7.80
```

```
Average WT = 4.80
```

Task 2: SJF – User Input + Averages (task2_sjf.c)

Task 2 Objective

Extend the SJF base code to accept user input for n, AT, BT and compute Average TAT and Average WT. Same non-preemptive algorithm: at each decision point, pick the arrived process with smallest burst time.

Manual Calculation for n

P	AT	BT
1	0	6
2	2	4
3	3	2
4	5	1
5	7	3

t=0: Only P1 arrived -> run P1(BT=6) -> CT[0]=6
 t=6: Arrived: P2(4),P3(2),P4(1) -> min=P4(1) -> CT[3]=7
 t=7: Arrived: P2(4),P3(2) -> min=P3(2) -> CT[2]=9
 t=9: Arrived: P2(4),P5(3) -> min=P5(3) -> CT[4]=12
 t=12: Only P2(4) left -> run P2(4) -> CT[1]=16

TAT: 6-0=6 ; 16-2=14 ; 9-3=6 ; 7-5=2 ; 12-7=5

WT: 6-6=0 ; 14-4=10 ; 6-2=4 ; 2-1=1 ; 5-3=2

Avg TAT = (6+14+6+2+5)/5 = 33/5 = 6.60

Avg WT = (0+10+4+1+2)/5 = 17/5 = 3.40

```

1 #include <stdio.h>
2
3 int main() {
4
5     int n;
6
7     printf("Enter number of processes: ");
8     scanf("%d", &n);
9
10    int pid[n], at[n], bt[n];
11    int ct[n], tat[n], wt[n];
12    int done[n];
13    int i;
14
15    for (i = 0; i < n; i++) {
16        pid[i] = i + 1;
17        done[i] = 0;
18        printf("Enter Arrival Time and Burst Time for P%d: ",
19              pid[i]);
20        scanf("%d %d", &at[i], &bt[i]);
  
```

```

21
22     int completed = 0, current_time = 0;
23     int smallest, flag;
24
25     while (completed < n) {
26
27         smallest = -1;
28         flag = 0;
29
30         for (i = 0; i < n; i++) {
31
32             if (!done[i] && at[i] <= current_time) {
33
34                 if (smallest == -1 || bt[i] < bt[smallest]) {
35                     smallest = i;
36                     flag = 1;
37                 }
38             }
39         }
40
41         if (!flag) {
42             current_time++;
43             continue;
44         }
45
46         current_time += bt[smallest];
47         ct[smallest] = current_time;
48         done[smallest] = 1;
49         completed++;
50     }
51
52     for (i = 0; i < n; i++) {
53         tat[i] = ct[i] - at[i];
54         wt[i] = tat[i] - bt[i];
55     }
56
57     printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
58     for (i = 0; i < n; i++) {
59         printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
60             pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
61     }
62
63     float sum_tat = 0, sum_wt = 0;
64     for (i = 0; i < n; i++) {
65         sum_tat += tat[i];
66         sum_wt += wt[i];
67     }
68
69     printf("\nAverage TAT = %.2f\n", sum_tat / n);
70     printf("Average WT = %.2f\n", sum_wt / n);
71

```

```

72     return 0;
73 }

```

Listing 5: task2_sjf.c – SJF with User Input and Averages

Line-by-Line Explanation (New/Key Lines Only)

```
done[i] = 0;
```

Initialized inside the input loop (combined with reading AT/BT). Sets each process as “not done”. This must happen before the scheduling while loop.

```
while (completed < n) { smallest = -1; flag = 0;
```

Reset the search at every scheduling decision. We need to find the best process from scratch each time because new processes may have arrived.

```
if (!done[i] && at[i] <= current_time)
```

Only consider processes that: have arrived ($AT \leq \text{current time}$) AND have not yet finished. This is the key filter for SJF.

```
if (smallest == -1 || bt[i] < bt[smallest]) { smallest = i; }
```

The SJF selection criterion: update `smallest` whenever we find a process with a **shorter** burst time. After the inner for loop finishes, `smallest` holds the index of the shortest available job.

```
if (!flag) { current_time++; continue; }
```

CPU idle: no eligible process found yet. Advance clock and retry. This handles gaps like when P1 finishes at $t=6$ but no other process arrives until $t=7$.

Sample Run Output (n=5)

P	AT	BT	CT	TAT	WT
P1	0	6	6	6	0
P2	2	4	16	14	10
P3	3	2	9	6	4
P4	5	1	7	2	1
P5	7	3	12	5	2

Average TAT = 6.60

Average WT = 3.40

Task 3: SRTF – Preemptive SJF (task3_srtf.c)

Task 3 Objective

Implement Shortest Remaining Time First (SRTF) – the preemptive version of SJF. At EVERY clock tick, the scheduler checks if any newly arrived process has a shorter remaining time than the currently running one. If yes, it preempts the current process.

Manual Calculation for n

P	AT	BT
1	0	6
2	2	4
3	3	2
4	5	1
5	7	3

SRTF (check every tick for shortest remaining):

t=0: P1(rem=6) only -> run P1. rem[0]=5 at t=1

t=2: P2 arrives(rem=4). P1 rem=4=P2 rem=4 -> keep P1 (tie: running wins)

t=3: P3 arrives(rem=2) < P1(rem=3) -> PREEMPT P1, run P3

t=5: P3 done. CT[2]=5. P4 arrives(rem=1) < P1(rem=3) -> run P4

t=6: P4 done. CT[3]=6. P1(rem=3), P2(rem=4) -> run P1

t=7: P5 arrives(rem=3). P1 rem=2 < 3 -> keep P1

t=8: P1 done. CT[0]=8. P2(rem=4), P5(rem=3) -> run P5

t=11: P5 done. CT[4]=11. Only P2(rem=4) -> run P2

t=15: P2 done. CT[1]=15

TAT: 8-0=8 ; 15-2=13 ; 5-3=2 ; 6-5=1 ; 11-7=4

WT: 8-6=2 ; 13-4=9 ; 2-2=0 ; 1-1=0 ; 4-3=1

Avg TAT = (8+13+2+1+4)/5 = 28/5 = 5.60

Avg WT = (2+9+0+0+1)/5 = 12/5 = 2.40

```

1 #include <stdio.h>
2
3 int main() {
4
5     int n;
6
7     printf("Enter number of processes: ");
8     scanf("%d", &n);
9
10    int pid[n], at[n], bt[n];
11    int ct[n], tat[n], wt[n];
12    int remaining[n];
13    int done[n];
14    int i;
```

```
15
16     for (i = 0; i < n; i++) {
17         pid[i] = i + 1;
18         done[i] = 0;
19         printf("Enter Arrival Time and Burst Time for P%d: ",
20             pid[i]);
21         scanf("%d %d", &at[i], &bt[i]);
22         remaining[i] = bt[i];
23     }
24
25     int completed = 0, current_time = 0;
26     int shortest, flag;
27
28     while (completed < n) {
29
30         shortest = -1;
31         flag = 0;
32
33         for (i = 0; i < n; i++) {
34
35             if (!done[i] && at[i] <= current_time && remaining[i]
36                 > 0) {
37
38                 if (shortest == -1 || remaining[i] < remaining[
39                     shortest]) {
40                     shortest = i;
41                     flag = 1;
42                 }
43             }
44
45             if (!flag) {
46                 current_time++;
47                 continue;
48             }
49
50             remaining[shortest]--;
51             current_time++;
52
53             if (remaining[shortest] == 0) {
54                 ct[shortest] = current_time;
55                 done[shortest] = 1;
56                 completed++;
57             }
58
59             for (i = 0; i < n; i++) {
60                 tat[i] = ct[i] - at[i];
61                 wt[i] = tat[i] - bt[i];
62             }
```

```

63     printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
64     for (i = 0; i < n; i++) {
65         printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
66             pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
67     }
68
69     float sum_tat = 0, sum_wt = 0;
70     for (i = 0; i < n; i++) {
71         sum_tat += tat[i];
72         sum_wt  += wt[i];
73     }
74
75     printf("\nAverage TAT = %.2f\n", sum_tat / n);
76     printf("Average WT  = %.2f\n",  sum_wt  / n);
77
78     return 0;
79 }

```

Listing 6: task3_srtf.c – SRTF (Preemptive SJF) with User Input

Line-by-Line Explanation (Key SRTF-Specific Lines)

```
remaining[i] = bt[i];
```

Initializes remaining burst time equal to full burst time. Unlike SJF, SRTF tracks remaining time because the process may be preempted midway and need to resume later.

```
if (!done[i] && at[i] <= current_time && remaining[i] > 0)
```

SRTF eligibility check: not done, arrived, and still has work. The extra `remaining[i] > 0` guard prevents re-selecting a just-completed process.

```
if (shortest == -1 || remaining[i] < remaining[shortest])
```

The KEY difference from SJF: we compare **remaining** burst time (not original bt). This means at every tick, the process with the shortest remaining time is selected – even if that means preempting the current one.

```
remaining[shortest]-;
```

Execute ONE tick of the selected process. Decrease its remaining time by 1. This tick-by-tick approach is what makes SRTF preemptive – after each unit, we re-evaluate the entire ready queue.

```
current_time++;
```

Advance the simulated clock by exactly 1 unit.

```
if (remaining[shortest] == 0) {
```

After decrementing, check if the process just finished. If remaining reached 0, record

completion time and mark done.

OS Concept – SRTF vs SJF

SJF (non-preemptive): Once a process starts, it runs to completion. Simpler. Higher average WT.

SRTF (preemptive): Every clock tick, the process with minimum remaining time runs. More context switches but lower average WT. SRTF gives the **optimal (minimum) average waiting time** among all scheduling algorithms.

Task 4: Round Robin TQ=4 – User Input (task4_rr_tq4.c)

Task 4 Objective

Implement RR with TQ=4 with user input for n, AT, BT. Calculate and display Average TAT and Average WT. Same algorithm structure as the given base code but with hardcoded TQ=4 and user-input data.

Manual Calculation for n

P	AT	BT
1	0	5
2	1	3
3	2	6
4	4	2
5	6	4

RR Pass (scan array, give each arrived process up to TQ=4):

Pass 1:

P1(rem=5>4): run 4, rem[0]=1, t=4
 P2(rem=3<=4): done, CT[1]=7, t=7
 P3(rem=6>4): run 4, rem[2]=2, t=11
 P4(rem=2<=4): done, CT[3]=13, t=13
 P5(rem=4<=4): done, CT[4]=17, t=17

Pass 2:

P1(rem=1<=4): done, CT[0]=18, t=18
 P3(rem=2<=4): done, CT[2]=20, t=20

CT: P1=18, P2=7, P3=20, P4=13, P5=17

TAT: 18-0=18; 7-1=6; 20-2=18; 13-4=9; 17-6=11

WT: 18-5=13; 6-3=3; 18-6=12; 9-2=7; 11-4=7

Avg TAT = (18+6+18+9+11)/5 = 62/5 = 12.40

Avg WT = (13+3+12+7+7)/5 = 42/5 = 8.40

```

1 #include <stdio.h>
2
3 int main() {
4
5     int n;
6
7     printf("Enter number of processes: ");
8     scanf("%d", &n);
9
10    int pid[n], at[n], bt[n];
11    int ct[n], tat[n], wt[n];
12    int remaining[n];
13    int i;
14
15    for (i = 0; i < n; i++) {

```

```

16     pid[i] = i + 1;
17     printf("Enter Arrival Time and Burst Time for P%d: ",
18           pid[i]);
19     scanf("%d %d", &at[i], &bt[i]);
20     remaining[i] = bt[i];
21 }
22
23 int tq = 4;
24 int completed = 0;
25 int current_time = 0;
26
27 while (completed < n) {
28     int executed = 0;
29
30     for (i = 0; i < n; i++) {
31
32         if (remaining[i] > 0 && at[i] <= current_time) {
33
34             executed = 1;
35
36             if (remaining[i] > tq) {
37                 current_time += tq;
38                 remaining[i] -= tq;
39             }
40             else {
41                 current_time += remaining[i];
42                 ct[i] = current_time;
43                 remaining[i] = 0;
44                 completed++;
45             }
46         }
47     }
48
49     if (!executed) {
50         current_time++;
51     }
52 }
53
54 for (i = 0; i < n; i++) {
55     tat[i] = ct[i] - at[i];
56     wt[i] = tat[i] - bt[i];
57 }
58
59 printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
60 for (i = 0; i < n; i++) {
61     printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
62           pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
63 }
64
65 float sum_tat = 0, sum_wt = 0;

```

```
66     for (i = 0; i < n; i++) {
67         sum_tat += tat[i];
68         sum_wt  += wt[i];
69     }
70
71     printf("\nAverage TAT = %.2f\n", sum_tat / n);
72     printf("Average WT  = %.2f\n",  sum_wt  / n);
73
74     return 0;
75 }
```

Listing 7: task4_rr_tq4.c – Round Robin TQ=4 with User Input and Averages

Line-by-Line Explanation

```
int tq = 4;
```

Time Quantum is fixed at 4. In a general RR program you could ask this from the user too. Here it is hardcoded as specified by the task.

```
remaining[i] = bt[i];
```

Done inside the input loop. Each process starts with its full burst time as remaining. This array is what the scheduler decrements.

```
int executed = 0;
```

Declared INSIDE the while loop. This resets to 0 at the start of every full pass through all processes. If after a full pass no process ran, we advance the clock.

```
if (remaining[i] > tq) { current_time += tq; remaining[i] -= tq; }
```

Process cannot finish this round. Give it exactly TQ=4 time units. It will get another chance in the next pass.

```
else { current_time += remaining[i]; ct[i] = current_time;
remaining[i] = 0; completed++; }
```

Process finishes in this quantum. Run for its exact remaining time. Record CT. Set remaining to 0 so it is never selected again.

Sample Output (n=5, TQ=4)

P	AT	BT	CT	TAT	WT
P1	0	5	18	18	13
P2	1	3	7	6	3
P3	2	6	20	18	12
P4	4	2	13	9	7
P5	6	4	17	11	7

Average TAT = 12.40

Average WT = 8.40

Task 5: RR Version 1 – Analysis of No-Queue Problem

Task 5 Objective

Understand WHY the Version-1 RR (from Tasks 3 and 4) is NOT a true Round Robin. Document the limitations of the simple array-scan approach vs a proper ready queue.

What is Wrong with Version 1?

Version 1 Limitations

1. **No FIFO Ready Queue:** Real Round Robin uses a queue. When a process is preempted (quantum expires), it goes to the **back of the queue**. Newly arrived processes also join at the back. Version 1 ignores arrival order entirely.
2. **Array-index bias:** The for loop always starts at $i=0$. This means P1 (index 0) always gets checked first in every pass, even if P3 joined the ready queue before P1's turn. In a real RR, position in the queue – not array index – determines who runs next.
3. **Incorrect interleaving:** Suppose P1 runs for TQ and doesn't finish. P2 then arrives. In real RR: queue order = [P2, P1]. Version 1 will check P1 again ($i=0$) before P2 ($i=1$), so P1 runs again before P2 – wrong.
4. **When TQ matters:** With very large TQ ($TQ \geq \max BT$), all processes finish in one pass and the bug is invisible. The bug shows when processes need multiple quanta and arrive at different times.

Concrete Example of the Bug

P	AT	BT	TQ=2
1	0	4	
2	1	2	

Version 1 (array scan):

```
t=0: P1 arrives, run P1 for 2 -> rem[0]=2, t=2
t=2: scan i=0: P1(rem=2) -> run P1 for 2 -> done, CT[0]=4
      scan i=1: P2(rem=2) -> run P2 for 2 -> done, CT[1]=6
      Order: P1->P1->P2 (P1 ran twice before P2 got any CPU!)
```

Real RR with Queue:

```
t=0: P1 arrives -> Queue: [P1]
      Run P1 for 2 -> rem=2. P2 arrived at t=1, enqueue -> Queue: [P2, P1]
t=2: Run P2 for 2 -> done. CT[1]=4 -> Queue: [P1]
t=4: Run P1 for 2 -> done. CT[0]=6
      Order: P1->P2->P1 (correct round-robin)
```

The code for Version 1 is identical to Task 4 with detailed comments added. See `task5_rr_v1_noqueue.c`.

Task 6: RR Version 2 – With Proper Ready Queue (task6_rr_v2_queue.c)

Task 6 Objective

Implement a REAL Round Robin scheduler using a proper FIFO ready queue. When a process uses its quantum and is not done, it goes to the BACK of the queue. Newly arrived processes are also added to the queue at the correct time.

Manual Calculation for n

P	AT	BT	
1	0	5	TQ = 4
2	1	3	
3	2	6	
4	4	2	
5	6	4	

Queue Trace:

t=0: P1 arrives -> Queue: [P1]. Run P1 for 4 -> rem[0]=1, t=4
 P2(t=1), P3(t=2), P4(t=4) arrived during run -> enqueue
 P1 not done -> re-enqueue P1 at back
 Queue: [P2, P3, P4, P1]

t=4: Dequeue P2, run for 3 -> done, CT[1]=7, t=7
 P5(t=6) arrived -> Queue: [P3, P4, P1, P5]

t=7: Dequeue P3, run for 4 -> rem[2]=2, t=11
 P3 re-enqueued -> Queue: [P4, P1, P5, P3]

t=11: Dequeue P4, run for 2 -> done, CT[3]=13, t=13
 Queue: [P1, P5, P3]

t=13: Dequeue P1, run for 1 -> done, CT[0]=14, t=14
 Queue: [P5, P3]

t=14: Dequeue P5, run for 4 -> done, CT[4]=18, t=18
 Queue: [P3]

t=18: Dequeue P3, run for 2 -> done, CT[2]=20, t=20

TAT: 14-0=14; 7-1=6; 20-2=18; 13-4=9; 18-6=12

WT: 14-5=9; 6-3=3; 18-6=12; 9-2=7; 12-4=8

Avg TAT = (14+6+18+9+12)/5 = 59/5 = 11.80

Avg WT = (9+3+12+7+8)/5 = 39/5 = 7.80

```

1 #include <stdio.h>
2
3 #define MAX 100
4
5 int queue[MAX];
6 int front = 0, rear = 0;
7
8 void enqueue(int val) {

```

```
9     queue[rear++] = val;
10 }
11
12 int dequeue() {
13     return queue[front++];
14 }
15
16 int isEmpty() {
17     return front == rear;
18 }
19
20 int main() {
21
22     int n;
23     printf("Enter number of processes: ");
24     scanf("%d", &n);
25
26     int pid[n], at[n], bt[n];
27     int ct[n], tat[n], wt[n];
28     int remaining[n];
29     int in_queue[n];
30     int i;
31
32     for (i = 0; i < n; i++) {
33         pid[i] = i + 1;
34         in_queue[i] = 0;
35         printf("Enter Arrival Time and Burst Time for P%d: ",
36             pid[i]);
37         scanf("%d %d", &at[i], &bt[i]);
38         remaining[i] = bt[i];
39     }
40
41     int tq = 4;
42     int completed = 0;
43     int current_time = 0;
44
45     for (i = 0; i < n; i++) {
46         if (at[i] == 0) {
47             enqueue(i);
48             in_queue[i] = 1;
49         }
50     }
51
52     if (isEmpty()) {
53         int min_at = at[0];
54         for (i = 1; i < n; i++) {
55             if (at[i] < min_at) min_at = at[i];
56         }
57         current_time = min_at;
58         for (i = 0; i < n; i++) {
59             if (at[i] == current_time) {
```

```

59         enqueue(i);
60         in_queue[i] = 1;
61     }
62 }
63 }
64
65 while (completed < n) {
66
67     if (isEmpty()) {
68         current_time++;
69         for (i = 0; i < n; i++) {
70             if (!in_queue[i] && remaining[i] > 0 && at[i] <=
71                 current_time) {
72                 enqueue(i);
73                 in_queue[i] = 1;
74             }
75             continue;
76         }
77
78         int curr = dequeue();
79         int run_time = (remaining[curr] > tq) ? tq : remaining
80             [curr];
81         int start_time = current_time;
82         current_time += run_time;
83         remaining[curr] -= run_time;
84
85         for (i = 0; i < n; i++) {
86             if (!in_queue[i] && remaining[i] > 0
87                 && at[i] > start_time && at[i] <= current_time)
88                 {
89                     enqueue(i);
90                     in_queue[i] = 1;
91                 }
92
93             if (remaining[curr] == 0) {
94                 ct[curr] = current_time;
95                 completed++;
96             }
97             else {
98                 enqueue(curr);
99             }
100         }
101
102         for (i = 0; i < n; i++) {
103             tat[i] = ct[i] - at[i];
104             wt[i] = tat[i] - bt[i];
105         }
106
107         printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");

```



```
107     for (i = 0; i < n; i++) {
108         printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
109             pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
110     }
111
112     float sum_tat = 0, sum_wt = 0;
113     for (i = 0; i < n; i++) {
114         sum_tat += tat[i];
115         sum_wt += wt[i];
116     }
117
118     printf("\nAverage TAT = %.2f\n", sum_tat / n);
119     printf("Average WT = %.2f\n", sum_wt / n);
120
121     return 0;
122 }
```

Listing 8: task6_rr_v2_queue.c – RR with Proper Ready Queue

Line-by-Line Explanation

```
#define MAX 100
```

Preprocessor constant for the maximum queue size. The queue array will hold up to 100 process indices. This is the backing storage for our FIFO queue.

```
int queue[MAX]; int front = 0, rear = 0;
```

Global array implements the queue. **front** = index of next item to dequeue (read). **rear** = index where next item will be enqueued (written). When `front == rear`, queue is empty.

```
void enqueue(int val) { queue[rear++] = val; }
```

Adds a process index `val` at the back of the queue, then increments `rear`. This is $O(1)$.

```
int dequeue() { return queue[front++]; }
```

Removes and returns the process index at the front of the queue, then increments `front`. The process at the front is the one that has waited longest (FIFO order). Also $O(1)$.

```
int isEmpty() { return front == rear; }
```

Returns 1 (true) if queue is empty (front and rear at same position), 0 otherwise.

```
int in_queue[n];
```

Boolean array. `in_queue[i] = 1` means process `i` is currently inside the ready queue. Prevents adding the same process twice. Set to 1 on enqueue; NOT reset on dequeue (this is intentional – once a process is put back, we don't need to re-add it from the

arrival check again since it's managed by re-enqueue).

```
for (i=0; i<n; i++) if (at[i]==0) { enqueue(i); in_queue[i]=1; }
```

Before starting the main loop, add all processes that arrive at time 0 to the queue. This bootstraps the ready queue.

```
if (isEmpty()) { ... find min_at, set current_time ... }
```

If no process arrived at $t=0$, find the first arrival time and jump the clock there. Then enqueue those first-arriving processes.

```
if (isEmpty()) { current_time++; ... enqueue newly arrived ...  
continue; }
```

During scheduling: if the queue is empty but there are still uncompleted processes, the CPU is idle. Advance clock by 1, check if any process has now arrived, enqueue them, and retry.

```
int curr = dequeue();
```

Get the next process to run from the FRONT of the ready queue. This is the process that has been waiting the longest among those in the queue – true FIFO Round Robin behavior.

```
int run_time = (remaining[curr] > tq) ? tq : remaining[curr];
```

Ternary operator: if the process needs more than TQ, run for TQ. Otherwise, run for its remaining time (it will finish). This replaces the if/else from Version 1.

```
int start_time = current_time; current_time += run_time;  
remaining[curr] -= run_time;
```

Record when we started (to check which processes arrived during this quantum), advance the clock, and update remaining burst time.

```
for (i=0; i<n; i++) { if (!in_queue[i] && ... at[i] > start_time &&  
at[i] <= current_time) { enqueue(i); } }
```

After the current process's quantum, check if any NEW processes arrived during that quantum (between `start_time` and `current_time`). Add them to the queue. They arrive AFTER the current process started, so they go behind it.

```
if (remaining[curr] == 0) { ct[curr] = current_time; completed++; }
```

Process finished. Record completion time.

```
else { enqueue(curr); }
```

Process NOT finished. Put it at the **BACK** of the ready queue. It will run again after all currently queued processes get their turn. This is the core of Round Robin.

Version 2 Output (n=5, TQ=4)

P	AT	BT	CT	TAT	WT
P1	0	5	14	14	9
P2	1	3	7	6	3
P3	2	6	20	18	12
P4	4	2	13	9	7
P5	6	4	18	12	8

Average TAT = 11.80

Average WT = 7.80

Quick Reference – Quiz Cheat Sheet

Formulas

- CT = time when process finishes execution
- $TAT = CT - AT$ (Turnaround Time)
- $WT = TAT - BT$ (Waiting Time)
- $Avg\ TAT = \sum TAT_i / n$
- $Avg\ WT = \sum WT_i / n$

Algorithm Differences in Code

Algorithm	Selection Logic	Key Code Pattern
FCFS	Run in arrival order	Simple for loop, no selection needed
SJF	$bt[i] < bt[smallest]$	while loop + inner search + done[] array
SRTF	$remaining[i] < remaining[shortest]$	while loop + tick-by-tick + remaining[] array
RR V1	Array scan with TQ	while loop + executed flag + remaining[]
RR V2	Dequeue from FIFO	queue + enqueue/dequeue + re-enqueue if not done

Common Mistakes to Avoid

Things to Remember for the Quiz

1. **FCFS**: Must sort by AT or assume already sorted. If `current_time < at[i]`, jump clock forward (CPU idle gap).
2. **SJF**: Uses `done[]` array. Searches for minimum `bt[i]` among arrived and not-done. Non-preemptive = no `remaining[]`, runs to completion.
3. **SRTF**: Uses `remaining[]` array. Checks minimum `remaining[i]` every tick. Decrements by 1 per tick. Very slow for large burst times (tick-by-tick).
4. **RR**: `remaining[i] > tq` → run for tq, reduce remaining. Otherwise → run for remaining, record CT, set `remaining=0`.
5. **RR V1 vs V2**: V1 loops array from index 0 always (wrong order). V2 uses proper FIFO queue, processes go back to end of queue after quantum.
6. **Averages**: Always use `float` for sum variables. Use `%.2f` in `printf` to show 2 decimal places.

Files Summary

File	Description
task1_fcfs.c	FCFS, user input, averages
task2_sjf.c	SJF non-preemptive, user input, averages
task3_srtf.c	SRTF preemptive, user input, averages
task4_rr_tq4.c	RR TQ=4, user input, averages
task5_rr_v1_noqueue.c	RR V1 with analysis comments (no queue)
task6_rr_v2_queue.c	RR V2 with proper FIFO ready queue

```
gcc task1_fcfs.c      -o task1
gcc task2_sjf.c      -o task2
gcc task3_srtf.c      -o task3
gcc task4_rr_tq4.c    -o task4
gcc task5_rr_v1_noqueue.c -o task5
gcc task6_rr_v2_queue.c -o task6
```

Listing 9: Compilation commands

Good luck on your quiz!

Key formulas: $TAT = CT - AT \cdot WT = TAT - BT \cdot Avg = Sum/n$